Arduino Uno の AES をアセンブリで読む

C++ の「１行」を、CPU への「細かい指示」にする。

知る
CPU ・メモリの基礎

01

比べる
どこを、どう変更したか

02

読む
実際の命令を１行ずつ

03

対象：今回作成した measurement.S と添付の Standard_AES_Measurement.ino

まず、何を測っていたのか？

平文＝ AES への入力データ。 16 バイトの平文を、同じ値で 10 回測定します。

平文
16 バイト
→
初期鍵 XOR
Round 0
→
置き換え
SubBytes
→
並べ替え
ShiftRows
→
混合
MixColumns
→
鍵 XOR
Round 1

D5 を HIGH にする → この計算区間を測定 → LOW にする

測定用の「初期 XOR ＋ 第 1 ラウンド」
AES-128 の全 10 ラウンド暗号化ではありません。

原本： execute_aes_std() 、 loop() / measurement.S:285–292, 148–175

両方とも、最後は「機械語」になる

アセンブリでは、 CPU に行わせる小さな操作を人が明示します。



まとまった処理を書き、変換を任せる



読む・計算する・保存する、を具体的に書く

レジスタ・ SRAM ・フラッシュを区別する

Uno は 8 ビット CPU 。レジスタ 1 個には、通常 1 バイトを入れます。

レジスタ

CPU 内の小さな作業場所

r18 、 r19 など

SRAM

処理中に読み書きする場所

state 、 pt 、 round_keys

フラッシュ

プログラムや定数を置く場所

命令、 S-Box

state は、計算途中の 16 バイトのデータです。

X ・ Z は「メモリのどこを見るか」を示す、アドレス用のレジスタ組です。

「命令 → 対象 → 説明」の順で読む

AVR の 2 オペランド命令では、一般に左側が結果の置き場所です。

命令の読み方（コメントは説明用）

```
eor   r18, r19    ; XOR
```

eor
XOR を行う命令

r18, r19
r18 と r19 を使う

; 以降
人が読むコメント

意味： r18 ← r18 XOR r19 。 r19 の値は変えず、結果を r18 に入れます。

XOR は「同じなら 0、違えば 1」

2 つの数を、ビットごとに比較します。足し算の繰り上がりはありません。

データ	0 0 1 1 1 1 0 0	0x3C
鍵	1 0 1 0 0 1 0 1	0xA5
結果	1 0 0 1 1 0 0 1	0x99

C++ の ^ と、AVR の eor は、どちらもこの XOR を表します。

C++ の 1 行を、 4 つの命令にする

X は state の現在位置、 Z は選んだラウンド鍵の現在位置を指す前提です。

変更前 / C++

```
state[i] ^= round_keys[round][i];
```

```
ld   r18, X
ld   r19, Z+
eor  r18, r19
st   X+, r18
```

① state の 1 バイトを読む

② 鍵の 1 バイトを読む

③ XOR を計算する

④ 同じ state の位置へ書き戻す

原本 :111 / measurement.S:309–312 （ポインタ初期化・ループ部分を除いた抜粋）

X+ の「+」は、処理した後に住所を 1 進める

実際の最初の平文と初期鍵を、1 バイトだけ追ってみます。

SRAM のデータ

state[0] = 0x7A

鍵 [0] = 0x2B

→

CPU の作業場所

r18 = 0x7A

r19 = 0x2B

→

XOR の結果

r18 = 0x51

```
st X+, r18
```

state[0] に 0x51 を保存

その後、X は state[1] を指す

0x51 は初期 XOR の途中結果です。第 1 ラウンド終了後の先頭バイトは 0x1C です。

繰り返しは「回数を減らし、戻る」で作る

AddRoundKey は、このループで state の 16 バイトを処理します。

実際のコードから抜粋

```
    ldi r20, 16
1:  ld  r18, X
    ld  r19, Z+
    eor r18, r19
    st  X+, r18
    dec r20
    brne 1b
```

ldi r20, 16
残り回数を 16 にする

dec r20
残り回数を 1 減らす

brne 1b
0 でなければ、ラベル 1 に戻る

S-Box は、フラッシュから 1 バイト読む

S-Box は「入力値に対応する値を引く、 256 個の対応表」です。

変更前： pgm_read_byte(...)

C++ の関数形式で対応表を参照

変更後： lpm r19, Z

Z の指すフラッシュを読む命令

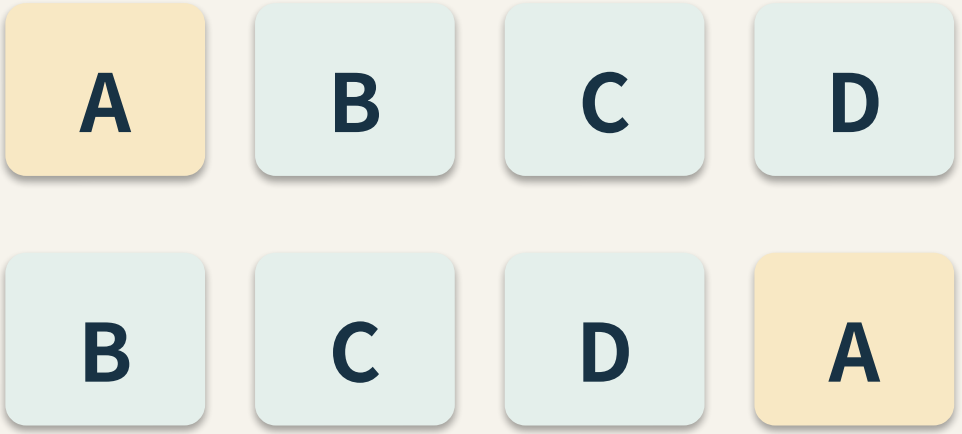
実際のコード / 1 バイト分

```
ld   r18, X
ldi  r30, lo8(sbox_std)
ldi  r31, hi8(sbox_std)
add  r30, r18
adc  r31, r1
lpm  r19, Z
st   X+, r19
```

先頭アドレス + 入力値 → 読む場所を決める → lpm で値を取り出す

並べ替えは「一時保存 → 読む → 書く」

例：行 1 の [state[1], state[5], state[9], state[13]] を左へ 1 つ回転。



最初の A を
レジスタへ退避し、
最後に書き戻す。

```
lds r18, state+1
lds r19, state+5
sts state+1, r19
```

r18 が C++ の temp に相当。
全行とも元と同じ並びになります。

「左シフト ＋ 必要な XOR 」で、列を混ぜる

AES の 2 倍演算は、 8 ビット内の左シフトと補正で計算できます。

変更前 / galois_mul2 の中身

```
(a << 1) ^ ((a & 0x80) ? 0x1B : 0x00)
```

XTIME r23,r24 の展開（ r25 = 0x1B ）

```
mov   r24, r23
lsl   r23
sbrc  r24, 7
eor   r23, r25
```

① 元の値を r24 に保存
② r23 を左に 1 ビットずらす
③ 元の最上位ビットが 0 なら次を飛ばす
④ 1 なら 0x1B と XOR して補正

列の式も同じ結果になる形へ整理。演算回数や波形が同じになる保証はありません。

原本 :galois_mul2() 、 MixColumns_Std() / measurement.S:364–413

32 ビットの値を、 4 つの 8 ビットに分ける

元の xorshift32 と同じ計算順とシードを維持しました。

元の計算 / 初期シード 12345

```
x ^= x << 13;
x ^= x >> 17;
x ^= x << 5;
```

```
lsl r22
rol r23
rol r24
rol r25
```

r18 r19 r20 r21

下位 → 上位の順に 4 バイト。
桁をまたぐ 1 ビットも引き継ぎます。
複製した r22–r25 をシフト。
lsl で出た 1 ビットを、
rol が次のバイトへ渡します。

条件を比較し、次に進む場所を選ぶ

大文字 S で開始・再開。完了後の S ではシードも初期化します。

受信済み文字を判定（最後のコメントは説明用）

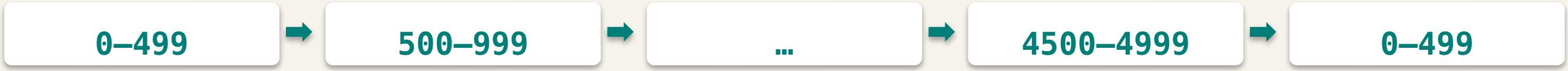
```
cpi   r24, 'S'
breq 2f
ret
2:   ; start / resume
```

cpi → 比較する

受信文字が S と同じか確認

breq → 等しければジャンプ

2f ：前方にあるラベル 2 へ進む



各バッチは S で進行。最後の矢印では、平文番号と乱数シードをリセット。

計算と進行管理は .S 、機器操作は Arduino へ

全体を動かすには、同じフォルダーにある .S と .ino の両方が必要です。

measurement.S
AES ・乱数・平文生成
繰り返し・再開・ログ順序

→

.ino の接続関数
アセンブリから
標準機能を呼ぶ窓口

→

Arduino Core
Serial / digitalWrite
pinMode / delay

アセンブリ側

```
call fw_trigger_high
```

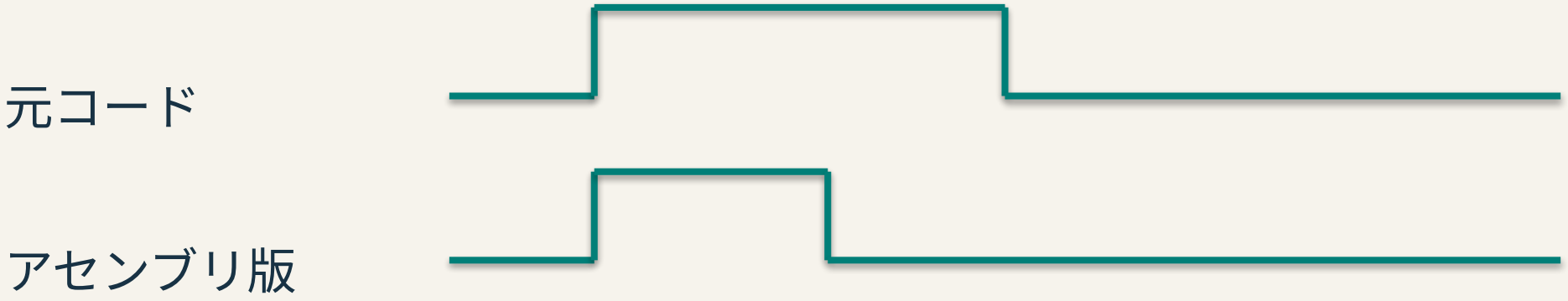
接続用 .ino 側

```
void fw_trigger_high() {
  digitalWrite(5, HIGH);
}
```

維持したのは、処理結果と測定の手順

命令列が変わるため、実行時間・トリガー幅・消費電力波形は変わります。

state へコピー | D5 HIGH | AES 計算 | D5 LOW | 10 ms 待機



模式図（実測値ではありません）。速度向上・実波形の一致は、今回の検証では主張しません。

元コードとの比較と、別実装で結果を確認

以下は AVR シミュレータでの検証です。物理的な Uno と実波形は未検証です。

5,500
平文
全 5000 ＋ リセット後 500

55,000
パルス
各平文を 10 回測定

60,500
待機要求
すべて delay(10)

一致：ログ全文、各パルスの入力・出力、乱数列、再開・リセット
追加確認：独立した Python 実装でも AES の結果を照合
実機用バイナリ：最初の平文で 10 パルス、待機の最短間隔は約 10.019 ms

根拠： tests/run.py 、 tests/check.py 、 build/asm-test/stats.json 、 build/uno/stats.json

「何を・どう・なぜ」で説明できるようにする

報告資料に使える説明例と、ソースを読む順番です。

何を変えたか	AES ・乱数生成・測定の進行管理を、 AVR アセンブリへ移した。
どう変えたか	C++ の処理を、読み込み・計算・保存・分岐の命令へ分解した。
なぜその形か	Uno の 8 ビット CPU で実行する操作を、明示できる形にした。

読む順番： execute_aes_std_asm → add_round_key_asm → 他の演算 → measurement_loop

最初は、この 12 種類を見分ける

意味はこの資料のコードでの使い方に合わせています。

`ldi`	数値を直接入れる
`ld / lds`	SRAM の値を読む
`st / sts`	SRAM に値を保存する
`lpm`	フラッシュの値を読む
`mov / movw`	値をコピーする
`eor`	XOR を計算する
`lsl / rol`	左シフト・桁を引き継ぐ
`cpi / cp`	値を比較する
`dec / inc`	1 減らす・1 増やす
`breq / brne`	等しい・等しくないとき分岐
`call / ret`	関数を呼ぶ・戻る
`push / pop`	値を一時退避・取り出す

ドット・ラベル・退避も、役割を分けて読む

CPU が実行する命令と、プログラムを組み立てる指定が混在します。

`.equ / .byte / .section`	定数・データ・配置先を指定する。 CPU の演算命令ではない。
`label: / 1b / 2f`	処理の目印。 b は後方、 f は前方の同じ番号のラベルを指す。
`lo8(...) / hi8(...)`	16 ビットのアドレスを、下位 8 ビット・上位 8 ビットに分ける。
`push / pop`	関数呼び出しで変わり得る値を退避し、逆順に取り戻す。
`r1 = 0 / r24 など`	Arduino とつなぐため、 AVR GCC の呼び出し規約に従う。

元コードとアセンブリを、同じ処理で探す

行番号は今回作成したファイルのものです。

元コードの処理	measurement.S の対応先	行
execute_aes_std	execute_aes_std_asm	285–292
AddRoundKey_Std	add_round_key_asm	297–315
SubBytes_Std	sub_bytes_asm	319–332
ShiftRows_Std	shift_rows_asm	336–361
MixColumns_Std	mix_columns_asm / XTIME	364–413
xorshift32	xorshift32_asm	230–281
setup / loop	measurement_setup / loop	40–212

資料：添付原本、 measurement.S 、接続用 .ino 、 tests/check.py 、検証結果 JSON

命令の定義： Microchip 「AVR Instruction Set Manual」／ GNU as 「AVR Dependent Features」